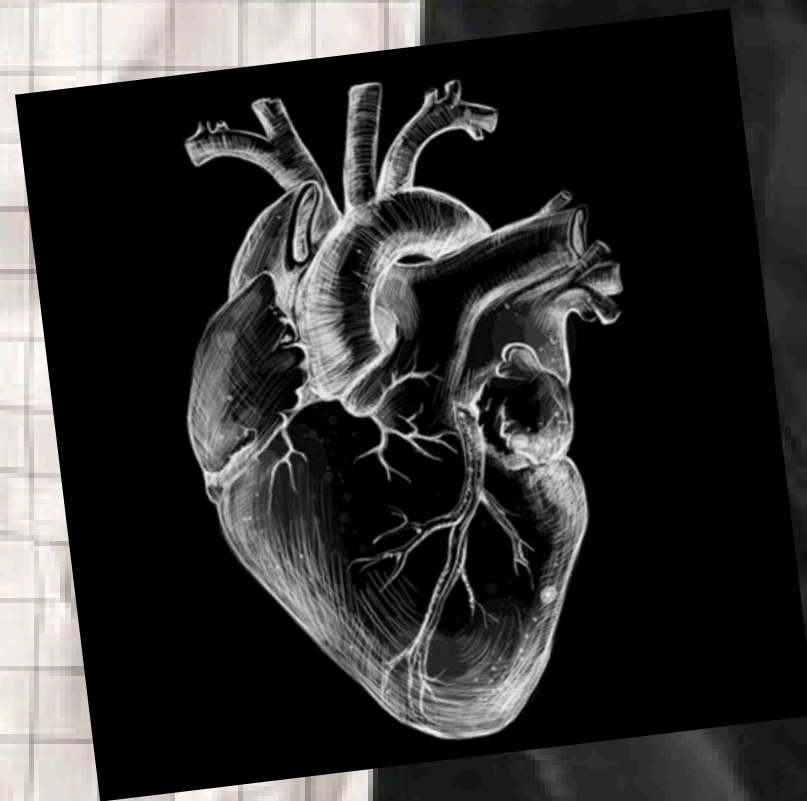




CLASIFICACIÓN DE ENFERMEDADES CARDÍACAS CON REDES NEURONALES



Introducción al proyecto

El dataset utilizado fue Heart Disease, obtenido del repositorio UCI Machine Learning Repository.

El objetivo del modelo es analizar distintas características médicas de un paciente y determinar si existe presencia de enfermedad cardíaca.

Se trata de un problema de clasificación binaria porque solamente existen dos posibles resultados:

- 0 = paciente sano.
- 1 = paciente con enfermedad cardíaca.



Importación de librerías

- NumPy
- Pandas
- Matplotlib y Seaborn
- Scikit-Learn
- TensorFlow y Keras

```
!pip install -q scikit-learn tensorflow matplotlib seaborn

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, roc_curve, auc

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

print(f'TensorFlow: {tf.__version__}')
print('✅ Listo')
```



Carga del dataset

Al cargar los datos, se separaron:

- Las variables predictoras o features.
- La variable objetivo o target.

Originalmente el dataset clasificaba la enfermedad cardíaca en cinco niveles:

- 0 = sano
- 1, 2, 3 y 4 = distintos grados de enfermedad

Para simplificar el problema, convertimos el target en binario:

- 0 = sano
- 1 = enfermo

De esta forma la red neuronal solo debe decidir si existe o no enfermedad cardíaca.



```
from ucimlrepo import fetch_ucirepo

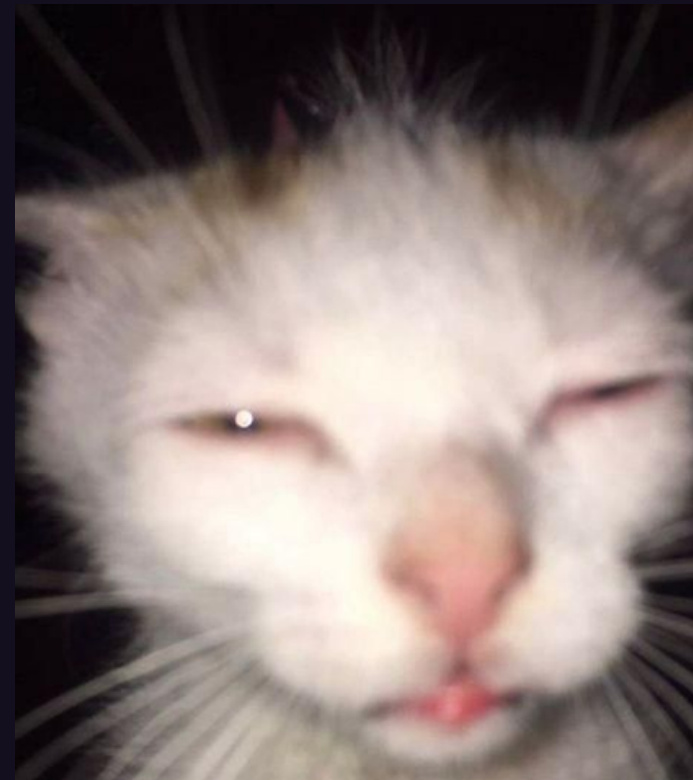
# Cargamos el dataset
heart_disease = fetch_ucirepo(id=45)

X_raw = heart_disease.data.features
y_raw = heart_disease.data.targets

# Armamos el DataFrame unificado
df = X_raw.copy()
df['target'] = y_raw.values

# El target original tiene valores 0,1,2,3,4
# Lo convertimos a binario: 0=sano, 1=enfermo
df['target'] = (df['target'] > 0).astype(int)

print(f'Shape: {df.shape}')
print(f'\nColumnas: {list(df.columns)}')
df.head()
```



Exploración de datos (EDA)

Distribución de clases:

Lo primero que se analizó fue la cantidad de pacientes sanos y enfermos.

Los resultados mostraron:

- Aproximadamente 54% de pacientes sanos.
- Aproximadamente 46% de pacientes enfermos.

Esto significa que el dataset está relativamente balanceado.

Por lo tanto, no fue necesario aplicar técnicas especiales para equilibrar las clases.




```
# 🛠 Reemplazá 'target' con el nombre de tu columna objetivo
TARGET_COL = 'target'

# Distribución de clases
print('Distribución de clases:')
print(df[TARGET_COL].value_counts())
print(f'\nPorcentajes:')
print(df[TARGET_COL].value_counts(normalize=True).mul(100).round(1).astype(str) + '%')
```

Distribución de clases:

target

0 164

1 139

Name: count, dtype: int64

Porcentajes:

target

0 54.1%

1 45.9%

Name: proportion, dtype: object



Valores nulos

También se revisó si existían datos faltantes.

Se encontraron valores nulos en:

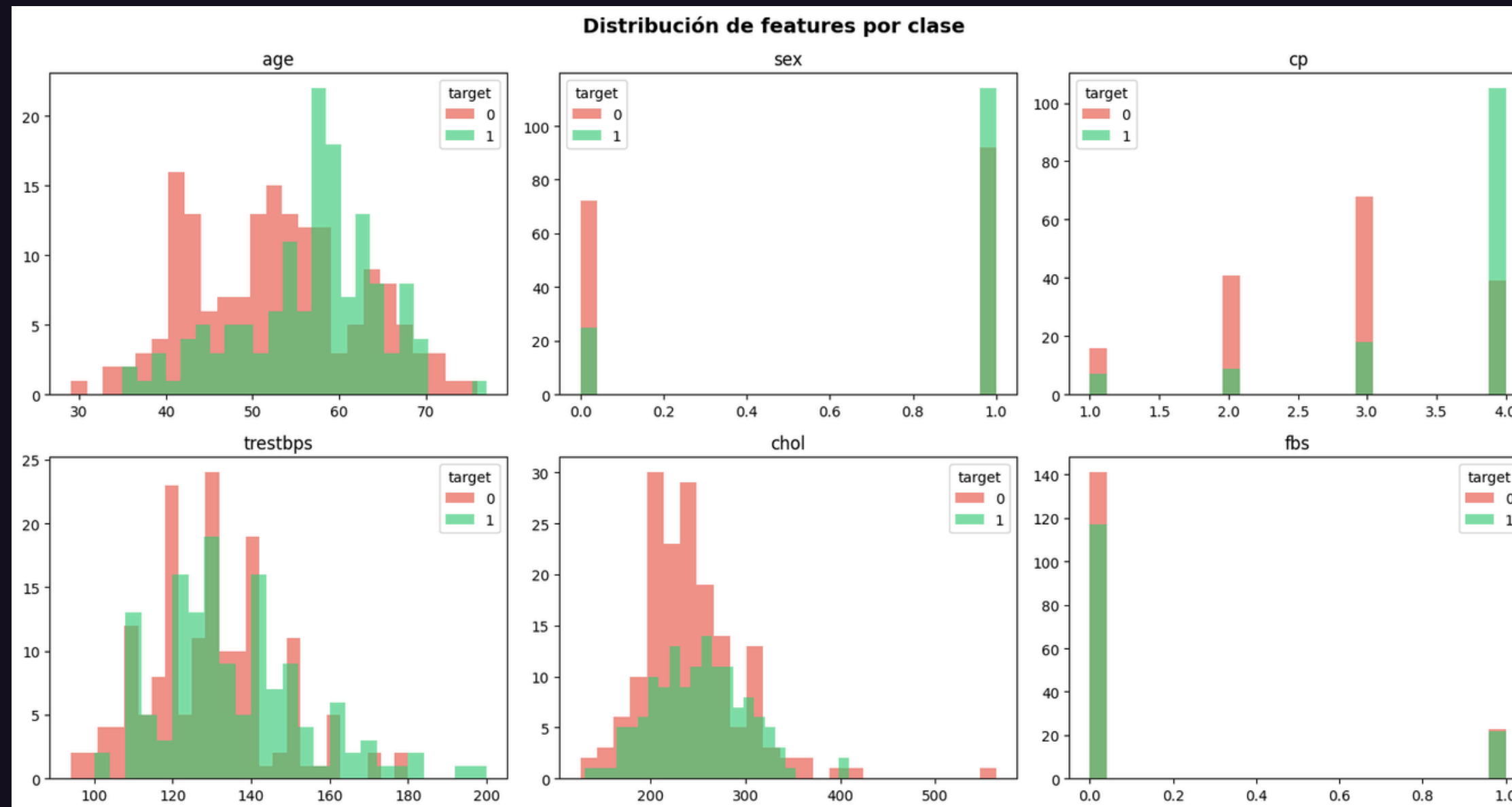
- ca → 4 valores faltantes.
- thal → 2 valores faltantes.

Como ambas son variables categóricas, se reemplazaron los valores faltantes utilizando la moda, es decir, el valor más frecuente de cada columna.

```
# Este dataset tiene una columna categórica: 'ca' y 'thal' pueden tener NaN
# Las rellenos con la moda (valor más frecuente)
X['ca'] = X['ca'].fillna(X['ca'].mode()[0])
X['thal'] = X['thal'].fillna(X['thal'].mode()[0])
```



Análisis de histogramas



Preprocesamiento de datos

- Separación de variables
- División Train-Test
- Normalización

```
# 🔧 Separar features y target
X = df.drop(TARGET_COL, axis=1)
y = df[TARGET_COL]
```

```
# Split train/test estratificado
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Normalización
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

print(f'Train: {X_train_s.shape} | Test: {X_test_s.shape}')

# Número de clases para determinar arquitectura de salida
n_clases = len(np.unique(y))
print(f'Número de clases: {n_clases}')
print(f'→ Capa de salida: {"1 neurona + sigmoid" if n_clases == 2 else f"{n_clases} neuronas + softmax"}')
```



Construcción de la red neuronal

- Capa de entrada
- Primera capa oculta
- Segunda capa oculta
- Drop outs
- Batch de normalización
- Capa de salida

```
model = keras.Sequential([
    layers.Input(shape=(n_features,)),

    # Capa oculta 1
    layers.Dense(32, activation='relu'),    # 🔧 cambiá 64 si necesitás
    layers.BatchNormalization(),
    layers.Dropout(0.3),

    # Capa oculta 2
    layers.Dense(16, activation='relu'),    # 🔧 cambiá 32 si necesitás
    layers.Dropout(0.2),

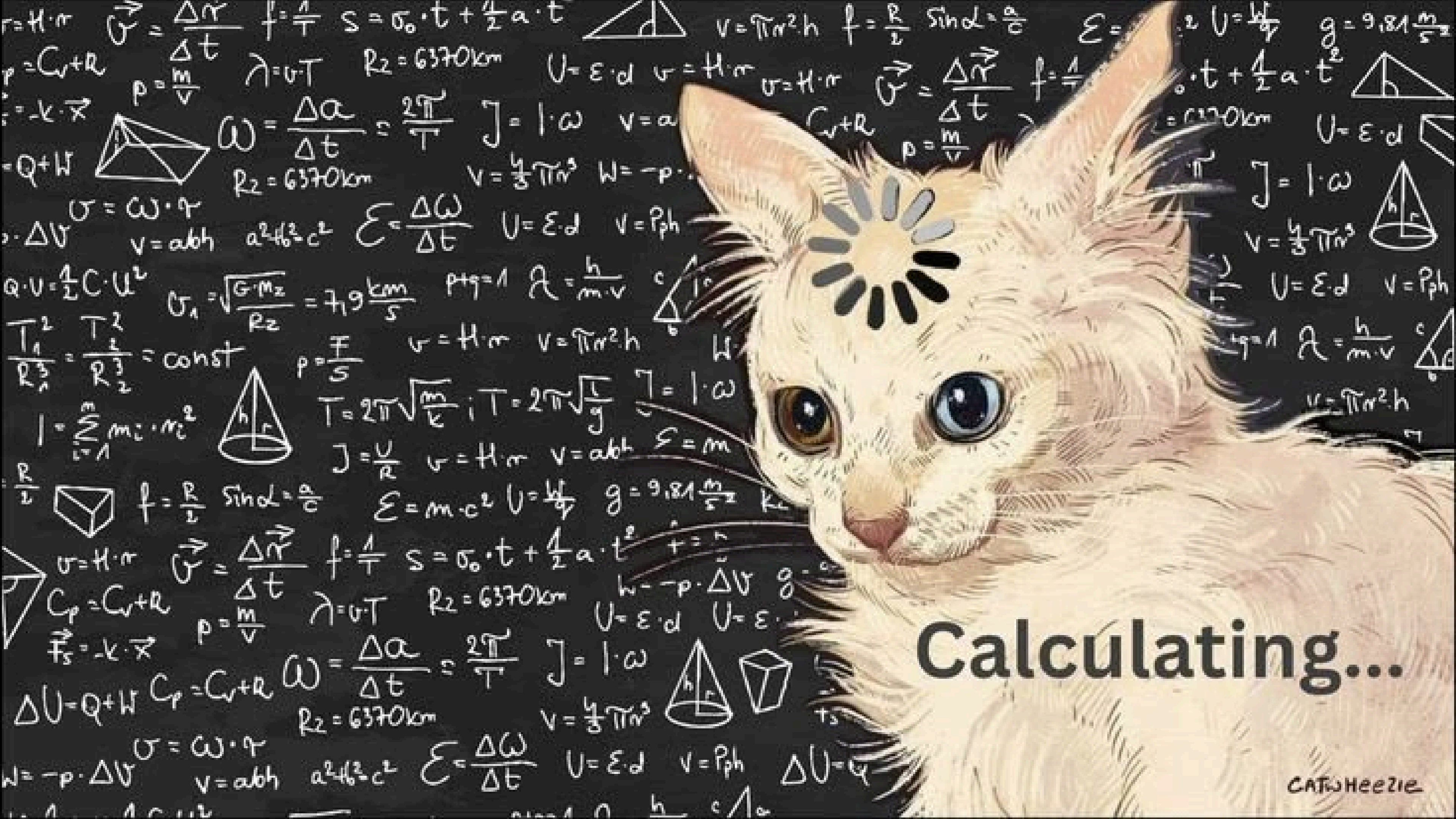
    # 🔧 Capa de salida:
    # → Binaria: layers.Dense(1, activation='sigmoid')
    # → Multiclase: layers.Dense(n_clases, activation='softmax')
    layers.Dense(1 if n_clases == 2 else n_clases,
                  activation='sigmoid' if n_clases == 2 else 'softmax')
])
```

```
# 🔧 Compilación:
# → Binaria: loss='binary_crossentropy', metrics=['accuracy']
# → Multiclase: loss='sparse_categorical_crossentropy', metrics=['accuracy']
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy' if n_clases == 2 else 'sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()
```

Se eligió hacer solo dos capas ocultas porque el dataset es pequeño y si se aumenta corre riesgo de overfitting





Calculating...

Entrenamiento

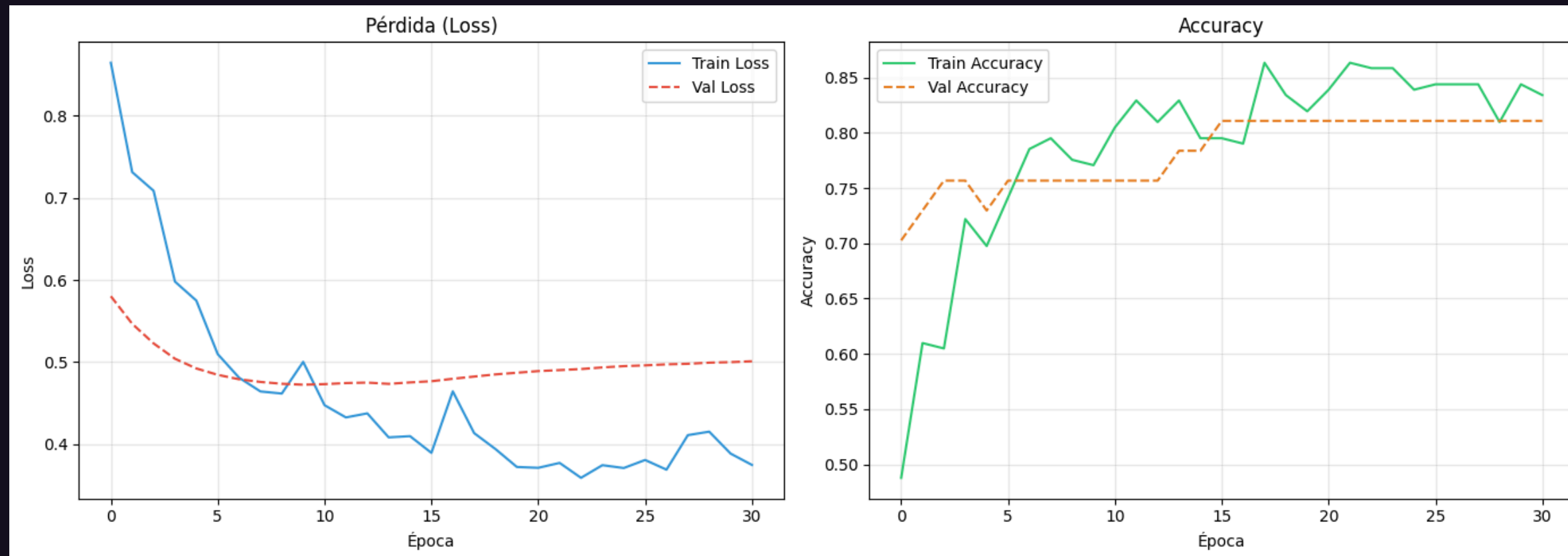
- Early stopping
- Reduce Learning Rate on Plateau
- Configuración

```
history = model.fit(  
    X_train_s, y_train,  
    epochs=100,      # 🔧 ajustar  
    batch_size=16,   # 🔧 ajustar  
    validation_split=0.15,  
    callbacks=callbacks,  
    verbose=1  
)  
  
print(f'\n✅ Entrenamiento finalizado en época {len(history.history["loss"])}')
```

```
callbacks = [  
    keras.callbacks.EarlyStopping(  
        monitor='val_accuracy',  
        patience=15,  
        restore_best_weights=True,  
        mode='max'  
    ),  
    keras.callbacks.ReduceLROnPlateau(  
        monitor='val_loss',  
        factor=0.5,  
        patience=7,  
        min_lr=1e-6  
    )  
]
```



Análisis de las curvas



Las curvas de entrenamiento y validación evolucionaron de forma similar.

No se observaron diferencias exageradas entre ambas.

Esto indica que el modelo aprendió correctamente y no presentó overfitting severo.



Evaluación del modelo

```
# Predicciones
y_prob = model.predict(X_test_s)

if n_clases == 2:
    y_prob = y_prob.flatten()
    y_pred = (y_prob >= 0.5).astype(int)
else:
    y_pred = np.argmax(y_prob, axis=1)

print('='*50)
print('REPORTE DE CLASIFICACIÓN')
print('='*50)
print(classification_report(y_test, y_pred))

loss, acc = model.evaluate(X_test_s, y_test, verbose=0)
print(f'Loss en test: {loss:.4f}')
print(f'Accuracy en test: {acc:.4f}')
```

2/2 ————— 0s 87ms/step

REPORTE DE CLASIFICACIÓN

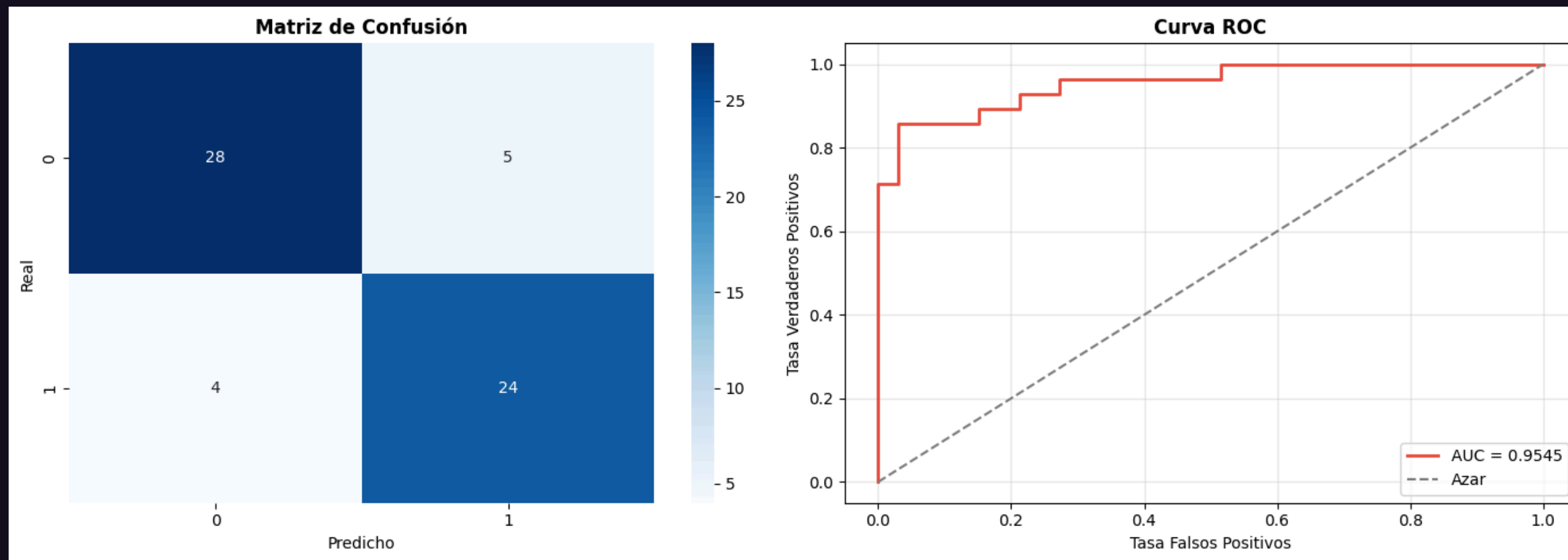
	precision	recall	f1-score	support
0	0.88	0.85	0.86	33
1	0.83	0.86	0.84	28
accuracy			0.85	61
macro avg	0.85	0.85	0.85	61
weighted avg	0.85	0.85	0.85	61

Loss en test: 0.3166

Accuracy en test: 0.8525



Matriz de confusión y curva ROC



Experimentos realizados

```
history_exp1 = modelo_exp1.fit(  
    X_train_s, y_train,  
    epochs=150,  
    batch_size=32,  
    validation_split=0.15,  
    callbacks=callbacks,  
    verbose=0  
)  
  
_, acc_exp1 = modelo_exp1.evaluate(X_test_s, y_test, verbose=0)  
print(f'Accuracy Experimento 1: {acc_exp1:.4f}')  
print(f'Accuracy Modelo base: {acc:.4f}')  
print(f'Diferencia: {(acc_exp1 - acc)*100:+.2f} puntos porcentuales')
```

```
Accuracy Experimento 1: 0.8033  
Accuracy Modelo base: 0.8525  
Diferencia: -4.92 puntos porcentuales
```

```
tf.random.set_seed(42)  
  
# EXPERIMENTO 1: Más neuronas  
modelo_exp1 = keras.Sequential([  
    layers.Input(shape=(n_features,)),  
    layers.Dense(128, activation='relu'),  
    layers.BatchNormalization(),  
    layers.Dropout(0.4),  
    layers.Dense(64, activation='relu'),  
    layers.Dropout(0.3),  
    layers.Dense(32, activation='relu'),  
    layers.Dense(1, activation='sigmoid')  
)  
  
modelo_exp1.compile(  
    optimizer=keras.optimizers.Adam(learning_rate=0.001), # \ ajustar  
    loss='binary_crossentropy' if n_clases == 2 else 'sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```



```
history_exp2 = modelo_exp2.fit(
    X_train_s, y_train,
    epochs=150,
    batch_size=32,
    validation_split=0.15,
    callbacks=callbacks,
    verbose=0
)
```

```
_, acc_exp2 = modelo_exp2.evaluate(X_test_s, y_test, verbose=0)
print(f'Accuracy Experimento 2: {acc_exp2:.4f}')
print(f'Accuracy Modelo base: {acc:.4f}')
print(f'Diferencia: {(acc_exp2 - acc)*100:+.2f} puntos porcentuales')
```

```
Accuracy Experimento 2: 0.2787
Accuracy Modelo base: 0.8525
Diferencia: -57.38 puntos porcentuales
```

```
# EXPERIMENTO 2: Learning rate más bajo
tf.random.set_seed(42)
```

```
modelo_exp2 = keras.Sequential([
    layers.Input(shape=(n_features,)),
    layers.Dense(64, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.3),
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(1, activation='sigmoid')
])
```

```
modelo_exp2.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0001), # antes era 0.001
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```



Registro de Experimentos

Experimento	Cambio realizado	Accuracy	¿Mejóro?
Modelo base	Arquitectura inicial	(0.8525)	—
Experimento 1	(Más neuronas)	(0.8033)	(no)
Experimento 2	(Learning rate más bajo)	(0.2787)	(no)



Conclusiones finales

Como conclusión, este proyecto demostró que una red neuronal simple puede predecir enfermedades cardíacas con una precisión cercana al 85%. Las variables más influyentes fueron el tipo de dolor de pecho (cp), la frecuencia cardíaca máxima (thalach) y el oldpeak. El mejor rendimiento se obtuvo con una arquitectura de 32 y 16 neuronas y una tasa de aprendizaje de 0.001. Además, se destacó la importancia del preprocesamiento, la normalización de datos y técnicas como Dropout y Early Stopping para evitar el overfitting. Como trabajo futuro, podrían probarse técnicas más avanzadas para mejorar aún más el modelo.





A horizontal strip of white paper with a torn, deckled edge is centered against a solid black background. The word "FIN" is printed in a bold, black, serif typeface on the white paper.

FIN